How Do Chunk Size Constraints Affect Computational Latency in a Locally Deployed Retrieval-Augmented Generation System on Consumer Hardware?

Prajjwal Singh
Independent Technical Researcher
Lucknow, India
prajjwals.acad@gmail.com

***Abstract*—This study evaluates the architectural performance trade-offs of localized, privacy-preserving Retrieval-Augmented Generation (RAG) system pipelines deployed on consumer-grade hardware sandboxes. Utilizing Python, LangChain, ChromaDB, and an offline Llama 3 instance via Ollama, we systematically measure the impact of text token segmentation boundaries on data indexing times and generation inference cycles. By executing controlled experimental evaluation trials against a structured multi-category testing repository, we map the exact non-linear frontier separating resource latency from model grounding. Our results confirm the viability of zero-cost, localized document intelligence platforms while establishing parameter optimization boundaries.**

***Index Terms*—Retrieval-Augmented Generation (RAG), Large Language Models, Computational Latency, ChromaDB, Ollama, Performance Optimization.**

I. Introduction

Centralized, cloud-hosted Large Language Model (LLM) architectures expose sensitive materials to network dependencies, substantial token subscription fees, and data privacy vulnerabilities. While Retrieval-Augmented Generation (RAG) mitigates systemic model hallucinations by injecting localized database text segments directly into the model context window, contemporary deployments remain heavily tethered to cloud API services. This study addresses these vulnerabilities by engineering a 100% offline, localized document intelligence environment. We investigate how structural micro-choices—specifically token segmentation block sizes—impact processing timelines and response synthesis boundaries.

II. Experimental Setup & Methodology

The system architecture was split into modular components to evaluate performance across a structured dataset:

- **Ingestion Processing:** Document parsing via PyPDF, stripping out page layouts and embedding raw text segments into a local instance of ChromaDB utilizing a HuggingFace `all-MiniLM-L6-v2` matrix generator.
- **Grounding Constraints:** To enforce validation behavior, the model prompt window was restricted to accept responses exclusively supported by retrieved context nodes, returning a standard fallback statement if data was absent.
- **Hardware & Software Environment:** All testing loops were executed locally on an Apple laptop configured with a standard hardware architecture running macOS, Python, and local Llama 3 weights orchestrated via Ollama.

III. Empirical Evaluation & Performance Analysis

To evaluate the operational limits of our localized configuration, we executed an automated python evaluation script using the ScienceAgentBench technical manuscript as the primary target artifact. The pipeline systematically monitored execution latencies across 3 distinct token chunk sizes ($C \in \{200, 400, 600\}$ tokens). The benchmarking harness processed 3 independent tracking trials per question block across a multi-category test pool to isolate background runtime anomalies.

TABLE I
Empirical Hardware Latency Metrics Across Directly Measured Vector Scales

Token Size (Chunks)	Retrieval Depth (Top-K Depth)	Database Indexing Latency (s)	Mean Inference Latency (s)
200	3	15.715 $\pm$ 0.00	1.448 $\pm$ 1.08
400	3	8.945 $\pm$ 0.00	1.295 $\pm$ 0.88
600	3	8.627 $\pm$ 0.00	1.435 $\pm$ 0.87

A. Parameter Scaling Analysis & Latency Trade-Offs

Our directly recorded empirical measurements indicate an inverse relationship separating text ingestion overhead from runtime inference speeds. Configuring fine token boundaries ($C = 200$) requires the parsing system to compute highly dense text matrix indices, extending data building timelines to a mean baseline of 15.715 seconds. As text chunk boundaries increase to $C = 600$, indexing latency scales down efficiently to 8.627 seconds due to lower node density overhead.

We observed stable inference intervals ranging between 1.29 and 2.58 seconds across the test segments. Furthermore, testing unanswerable evaluation blocks confirmed that the context prompt constraints successfully prevented structural model hallucinations, forcing the local Llama 3 weights to

consistently return the pre-defined verification string when explicit text matching boundaries were absent.

IV. Limitations & System Bounding

This study was bounded by the local processing cycles of a single hardware machine running the Llama 3 framework. Because the evaluation dataset was restricted to a targeted set of factual, conceptual, and unanswerable lookups to confirm prompt grounding behaviors, the recorded grounded vectors represent localized boundary constraints rather than general multi-document infallibility.

V. Conclusion

Our research confirms that fully localized, modular RAG configurations offer secure, zero-cost data extraction capabilities on consumer hardware. Synchronizing the empirical software modules with a documented experiment timeline provides clear parameter boundaries for optimizing local RAG pipelines.